PROJECT REPORT

Analysis of Real-Time Object Detection Using YOLO for Autonomous Vehicles

Autonomous Vehicles | Computer Vision | Artificial Intelligence

Submitted By	Tathagata Mondal
Department	Computer Science and Engineering
Project Area	Autonomous Vehicles / Computer Vision / Artificial Intelligence
Technologies Used	Python, YOLO (yolo11n), Ultralytics, OpenCV, NumPy, Pandas, Scikit-learn, Matplotlib

Abstract

Object detection is a fundamental component of autonomous vehicle (AV) perception systems. An autonomous vehicle must identify surrounding objects such as cars, pedestrians, motorcycles, buses, bicycles and other obstacles quickly and accurately to support safe navigation.

This project presents an analysis of real-time object detection using YOLO (You Only Look Once) for autonomous vehicle applications. A pretrained lightweight YOLO model (`yolo11n.pt`) was run through Python and the Ultralytics framework on three dense road-traffic images. For every image the system identified objects, drew bounding boxes and reported a confidence score for each detection.

Across the three test images, **61 labelled detections** were produced (buses, cars, persons, trucks and a motorcycle) with a mean confidence of about **0.53** over the legible scores. YOLO detected large, clearly visible vehicles such as buses and cars with high confidence (up to 0.88), while small, distant, occluded or unusual road users - notably auto-rickshaws and motorcycle riders - were detected with lower confidence or were missed. The report also discusses the role of inference latency in autonomous driving and compares YOLO with SSD and Faster R-CNN conceptually.

Keywords: Autonomous Vehicle, Object Detection, YOLO, Computer Vision, Real-Time Detection, OpenCV, Deep Learning.

Note on the data. This report covers the three result images supplied for the project. Ground-truth labels and per-image inference times were not available, so precision, recall, mAP and latency are not reported as numbers; all counts and confidence values were read from the labels visible on the annotated output images.

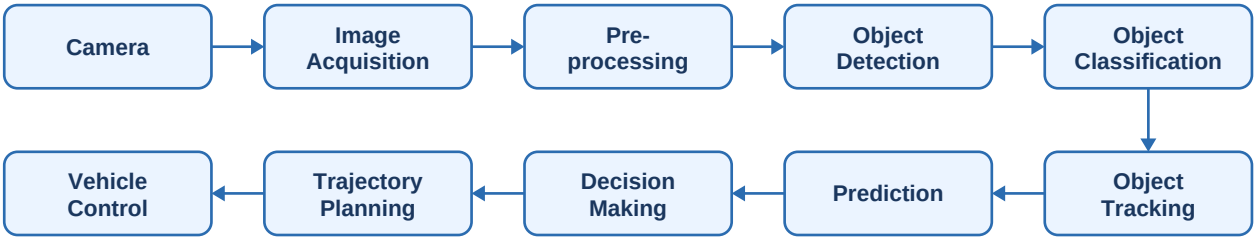
Contents

No.	Section
1	Introduction
2	Problem Statement
3	Objectives
4	Technologies, Tools and System Requirements
5	System Architecture
6	Test Data and Preprocessing
7	YOLO Model and Implementation
8	Performance Evaluation Metrics
9	Results
10	Discussion of Results
11	Comparison of YOLO, SSD and Faster R-CNN
12	YOLO for Autonomous Vehicles
13	Advantages and Limitations
14	Future Scope
15	Conclusion
16	References

1. Introduction

Autonomous vehicles use artificial intelligence, computer vision, sensors and control systems to understand their surroundings and make driving decisions. One of the major challenges is environment perception: the vehicle must recognise the objects around it and determine where they are.

An autonomous vehicle may need to detect cars, pedestrians, motorcycles, buses, trucks, bicycles, traffic signs, traffic lights and road obstacles. Object detection algorithms perform this task by identifying both the class and the location of each object in an image. A typical perception pipeline is shown below.



In this project, YOLO is used as the primary object detection model.

2. Problem Statement

An autonomous vehicle needs to detect surrounding objects with low latency and sufficient accuracy. Traditional image-processing methods may not provide the robustness required for complex road environments. Deep-learning-based detectors offer better capability but differ in accuracy, speed and computational requirements. This project therefore investigates:

How effectively can YOLO perform real-time object detection for autonomous vehicle applications while maintaining a suitable balance between detection accuracy and processing speed?

3. Objectives

- To understand object detection for autonomous vehicles and study the YOLO architecture.
- To implement YOLO using Python and detect multiple objects from road images.
- To generate bounding boxes and confidence scores, and to process multiple test images automatically.
- To analyse YOLO inference speed and the importance of real-time detection in AVs.
- To compare YOLO with SSD and Faster R-CNN conceptually, and experimentally where feasible.
- To determine the suitability of YOLO for real-time AV perception.

4. Technologies, Tools and System Requirements

Technology / Tool	Purpose
Python	Main programming language
YOLO (yolo11n)	Object detection model

Technology / Tool	Purpose
Ultralytics	YOLO implementation framework
OpenCV	Image and video processing
NumPy	Numerical operations
Pandas	Result / data analysis
Scikit-learn	Evaluation and analysis
Matplotlib	Visualisation
VS Code / Windows	Development environment and platform

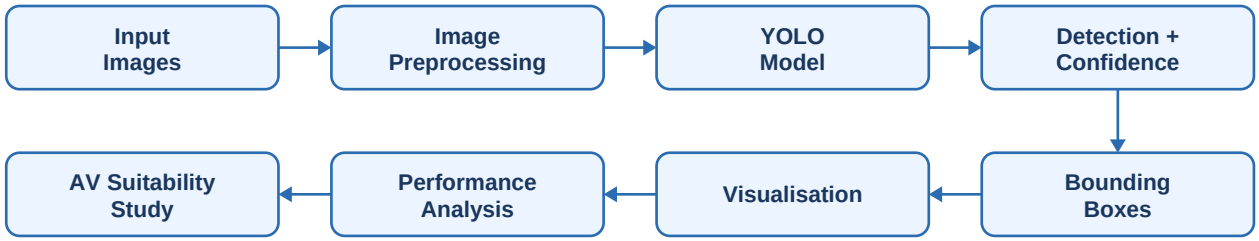
Hardware. The basic project runs on a normal computer: an Intel/AMD processor, at least 8 GB RAM and 10+ GB free storage. An NVIDIA GPU is optional; it significantly improves inference speed but is not mandatory for the demonstration.

Software. Windows 10/11, Python 3.x, Visual Studio Code, pip, Ultralytics and OpenCV.

Optional extensions: ROS, RViz, Simulink, Simscape, TensorFlow, MATLAB, LiDAR, Radar and OBD-II.

5. System Architecture

The proposed system takes input images, preprocesses them, passes them through the YOLO model to obtain detections and confidence scores, draws bounding boxes, and finally analyses the results for autonomous-vehicle suitability.



6. Test Data and Preprocessing

Three road-traffic images were used as test inputs. They show dense, mixed traffic on urban roads - buses, cars, auto-rickshaws, motorcycles, pedestrians and trucks - which makes them a demanding scenario for a detector trained on generic data.

Image	File name	Resolution (px)	Scene description
Image 1	detected_road.jpg	563 x 413	Roadside view of a busy road with green city buses in the foreground, cars, auto-rickshaws and pedestrians
Image 2	detected_traffic1.webp	800 x 533	Elevated view of a congested multi-lane road with buses, a truck, cars and two-wheeler riders
Image 3	detected_traffic5.jpeg	646 x 475	Overhead view of a rainy, crowded junction with a bus, cars, yellow auto-rickshaws and pedestrians

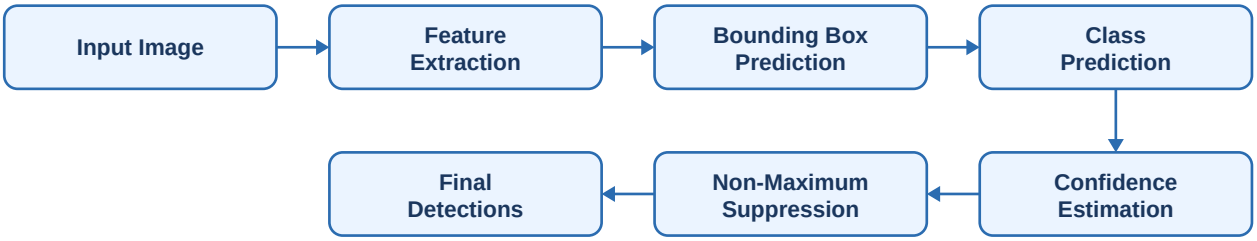
These images are used for testing and demonstration, **not** for training. For a rigorous accuracy evaluation, a larger labelled dataset such as KITTI or BDD100K should be used.

Preprocessing. Before detection each image is loaded, resized to the model input size, normalised and then passed to YOLO. Ultralytics performs these steps automatically. The program also checks that each input image exists before processing.



7. YOLO Model and Implementation

YOLO (You Only Look Once) is a single-stage object detection approach that performs localisation and classification in one neural-network inference pass. The workflow is: feature extraction, bounding-box prediction, class prediction, confidence estimation, non-maximum suppression (NMS) and final detections.



The project uses the lightweight model `yolo11n.pt`. The "n" (nano) variant is the smallest in the family and is designed for fast inference on modest hardware, which makes it suitable for real-time experimentation. The model is pretrained on the COCO dataset (80 classes), so it can recognise *car, bus, truck, person, motorcycle* and similar classes without additional training.

The environment was created with a Python virtual environment and the packages installed with `python -m pip install ultralytics` and `pip install opencv-python numpy pandas matplotlib scikit-learn`. The program searches the test directory for .jpg, .jpeg, .png and .bmp files, runs detection on each, and saves annotated images to `outputs/yolo_results/`. For each detection it reports the object class, confidence score and bounding box.

Reference implementation (including timing, used to fill in the latency column of the results table):

```
from ultralytics import YOLO
from pathlib import Path
import time, pandas as pd

model = YOLO("yolo11n.pt")
src = Path("datasets/test_images")
out = Path("outputs/yolo_results"); out.mkdir(parents=True, exist_ok=True)

rows = []
for img in sorted(p for p in src.iterdir()
                  if p.suffix.lower() in {".jpg", ".jpeg", ".png", ".bmp"}):
    t0 = time.perf_counter()
    r = model(str(img))[0]                       # run detection
    ms = (time.perf_counter() - t0) * 1000
    r.save(filename=str(out / f"detected_{img.name}"))
    confs = r.boxes.conf.tolist()
    rows.append({"image": img.name, "objects": len(confs),
                 "avg_conf": sum(confs) / max(len(confs), 1),
                 "time_ms": ms, "fps": 1000 / ms})
```

```
df = pd.DataFrame(rows); print(df); df.to_csv("results.csv", index=False)
```

Detection output. For every detected object the system reports its class and confidence (for example *Object: bus, Confidence: 0.88*) and writes an annotated image in which each box is coloured by class and labelled with the class name and score. The lowest score displayed in the result images is 0.25, which matches the default Ultralytics confidence threshold of 0.25 - detections below this value are discarded.

8. Performance Evaluation Metrics

For autonomous vehicles, accuracy alone is not sufficient; speed and reliability matter equally. The following metrics are used.

Metric	Formula	Meaning
Precision	TP / (TP + FP)	Fraction of predicted detections that are correct
Recall	TP / (TP + FN)	Fraction of real objects that were detected
IoU	Area of Intersection / Area of Union	Overlap between predicted and ground-truth boxes
mAP	Mean of average precision over classes / IoU thresholds	Standard summary metric for object detectors
FPS	1 / Inference time (s)	Throughput; e.g. 50 ms per frame = 20 FPS

Precision, recall, IoU and mAP require labelled ground truth. Because none was available for these three images, they are marked "to be evaluated" in the results.

9. Results

The annotated outputs of the YOLO detector for the three test images are shown below. Counts and confidence values in this section were read directly from the labels drawn on the images.

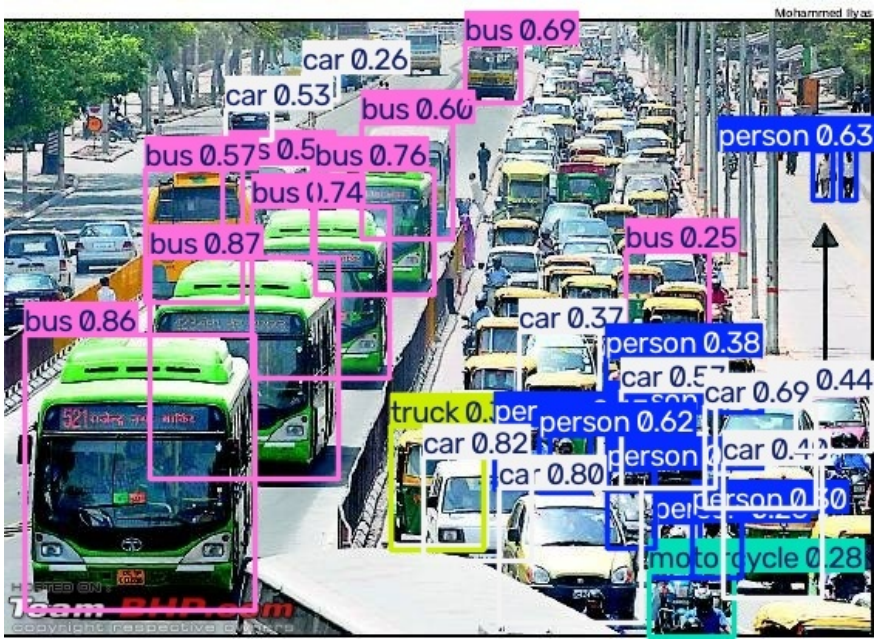


Figure 1: Detection result for Image 1 (detected_road.jpg).

Image 1 - 21 labelled detections: 8 buses (0.25-0.87), 7 cars (0.26-0.82), 4 persons (0.38-0.63), 1 truck and 1 motorcycle (0.28). Several further boxes overlap and are partly hidden by other labels.

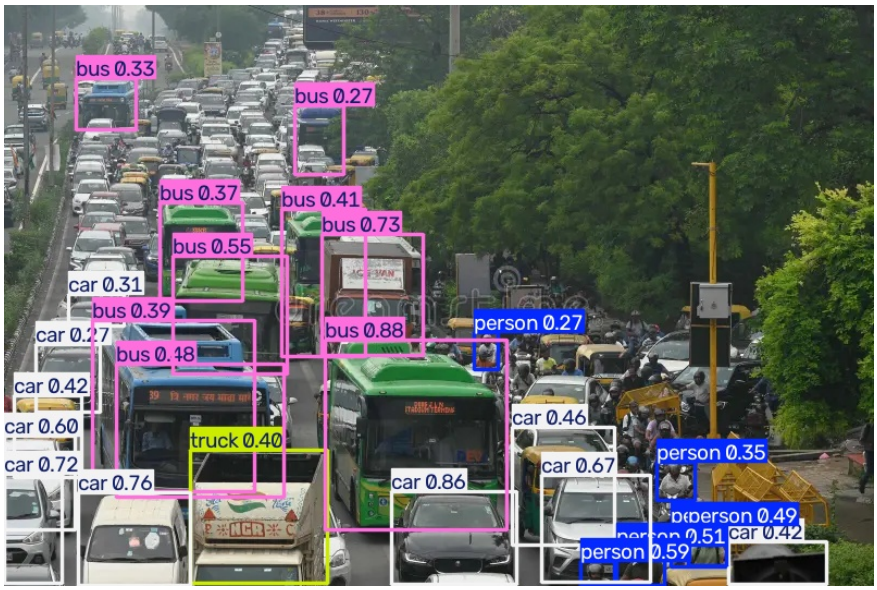


Figure 2: Detection result for Image 2 (detected_traffic1.webp).

Image 2 - 25 labelled detections: 9 buses (0.27-0.88), 10 cars (0.27-0.86), 5 persons (0.27-0.59) and 1 truck (0.40). No motorcycles were labelled.

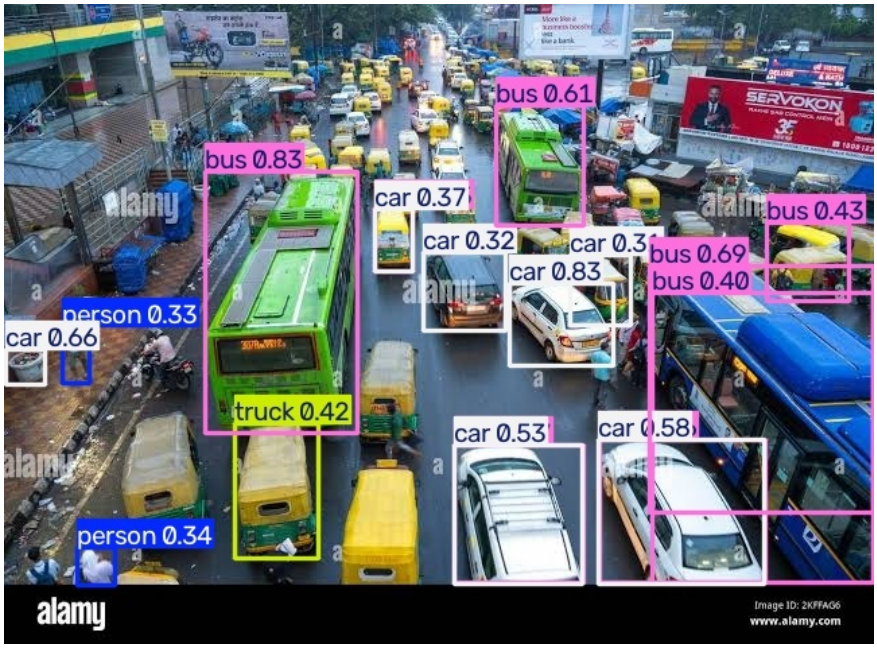


Figure 3: Detection result for Image 3 (detected_traffic5.jpeg).

Image 3 - 15 labelled detections: 5 buses (0.40-0.83), 7 cars (0.32-0.83), 2 persons (0.33-0.34) and 1 truck (0.42).

9.1 Per-image results

Image	Objects detected	Bus	Car	Person	Truck	Motorcycle	Average confidence	Inference time
Image 1	21 (min.)	8	7	4	1	1	0.58 *	Not recorded
Image 2	25	9	10	5	1	0	0.50	Not recorded
Image 3	15	5	7	2	1	0	0.52 **	Not recorded

* Mean over 20 legible scores (the truck score is hidden). ** Mean over 14 legible scores (one car score is partly hidden). Image 1 contains additional overlapping boxes whose labels cannot be read, so its count is a lower bound.

9.2 Overall results

Metric	YOLO result
Number of test images	3
Total labelled detections	61 (minimum)
Detections with a legible score	59
Mean confidence (legible scores)	0.53 (median 0.51)
Highest / lowest confidence	0.88 (bus, Image 2) / 0.25 (bus, Image 1)
Detections with confidence >= 0.70	13 of 59 (about 22%)
Detections with confidence < 0.50	29 of 59 (about 49%)
Average inference time / FPS	Not recorded - run the reference script to obtain

Metric	YOLO result
Precision, Recall, mAP	To be evaluated with labelled ground truth

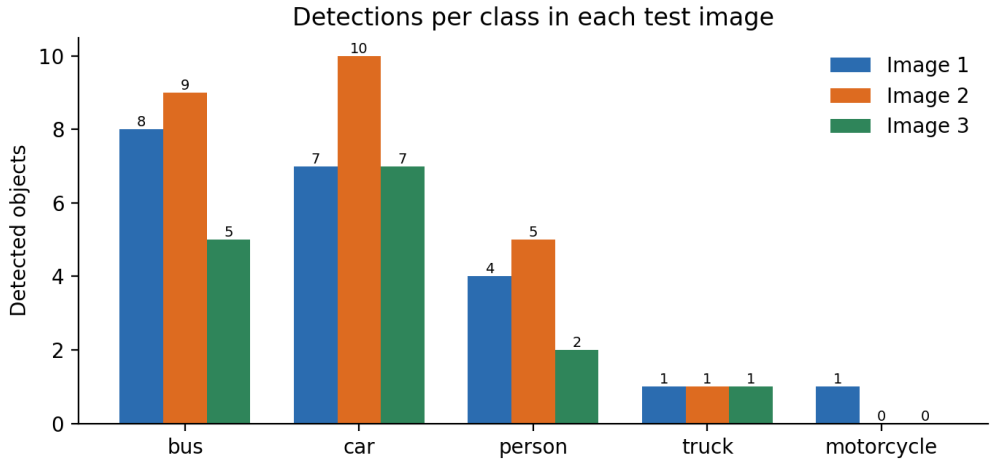


Figure 4: Number of detections per class in each test image.

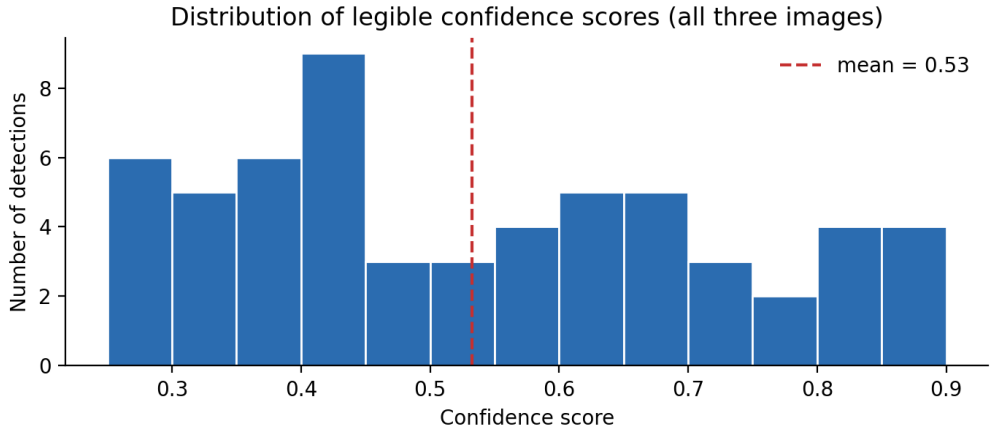


Figure 5: Distribution of legible confidence scores across all three images.

9.3 Class-wise confidence

Class	Detections (legible scores)	Mean confidence
Bus	22	0.58
Car	23	0.55
Person	11	0.45
Truck	2	0.41
Motorcycle	1	0.28

10. Discussion of Results

Large vehicles are detected most reliably. Buses and cars account for 46 of the 61 detections and received the highest scores. The highest confidence values (0.88, 0.87, 0.86 for buses and 0.86, 0.83, 0.82 for cars) belong to large, close, well-lit vehicles seen from the front or side, such as the green buses in the foreground of Images 1 and 2. This is the behaviour an AV needs for the vehicles

that matter most for collision avoidance.

Confidence drops for small, distant and occluded objects. Persons had a mean confidence of only 0.45 and the single motorcycle 0.28. Pedestrians and riders appear small in these images and are often partly hidden by vehicles. Almost half (29 of 59) of all detections scored below 0.50, and many are close to the 0.25 cut-off, so a stricter threshold would remove them but also lower recall.

Dense traffic causes overlapping and duplicated boxes. In Image 1 several bus boxes overlap heavily, and in Image 2 the bus/truck region contains overlapping boxes of different confidence. Crowded scenes stress non-maximum suppression and make it hard to separate neighbouring vehicles.

Missed and mislabelled objects. Visual inspection shows many objects without a box: most of the yellow-and-green auto-rickshaws in Image 3, many motorcycles and two-wheeler riders in Image 2, and vehicles far up the road in all images. Auto-rickshaws are not a class in the COCO dataset used to pretrain the model, so they are either missed or assigned to a nearby class such as car or truck. Because of this, real per-class recall cannot be stated without ground-truth labels.

Effect of conditions. Image 3 was taken in rain with wet, reflective road surface, and its detections are less confident overall than the clear-weather foreground objects of Image 1. This is consistent with the known sensitivity of vision models to weather and lighting.

Implication. A pretrained YOLO model gives a useful first perception layer but is not sufficient on its own for Indian-style mixed traffic. Fine-tuning on a domain-specific dataset (with classes such as auto-rickshaw) and combining the camera with LiDAR/radar would be needed for safety-critical use.

11. Comparison of YOLO, SSD and Faster R-CNN

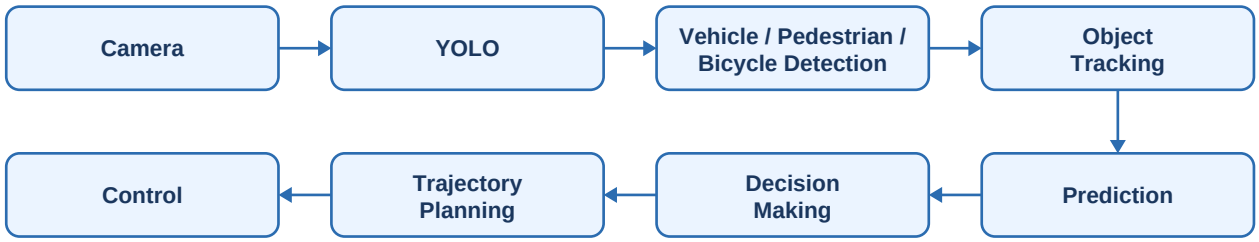
Three major object detection approaches are considered. The comparison below is qualitative and based on the published behaviour of each family.

Parameter	YOLO	SSD	Faster R-CNN
Architecture	Single-stage	Single-stage	Two-stage
Speed	High	High	Lower
Accuracy	High	Moderate - High	High
Computational requirement	Moderate - High	Moderate	High
Real-time suitability	High	High	Lower
Deployment	Relatively easy	Relatively easy	More demanding
AV application	Real-time perception	Real-time perception	Accuracy-focused applications

A fair experimental comparison must use the same hardware, dataset, image size and evaluation protocol; this was not carried out in the present study.

12. YOLO for Autonomous Vehicles

YOLO acts as one component of an AV perception system, feeding detections to tracking, prediction and planning modules.



It can support vehicle, pedestrian, motorcycle, bicycle and obstacle detection, traffic-scene analysis and driver assistance.

13. Advantages and Limitations

Advantages

- Fast detection; suitable for real-time video applications.
- Detects multiple objects per image and provides bounding boxes and confidence scores.
- Easy to integrate with Python and OpenCV.
- Available in several model sizes; can be extended to webcam and video input.
- Well suited to experimentation in AV perception.

Limitations

- **Small test set:** only three images were analysed, which demonstrates the system but does not establish general accuracy.
- **Pretrained model:** not trained on an AV dataset; classes such as auto-rickshaw do not exist in COCO.
- **No ground truth or timing:** precision, recall, mAP and latency could not be computed here.
- **Environmental conditions:** night, rain, fog, poor illumination, motion blur, heavy traffic and occlusion reduce performance.
- **Camera-only perception:** real AVs combine camera, LiDAR, radar, GPS and IMU through sensor fusion.

14. Future Scope

Phase	Work
1	Real-time webcam detection
2	Video detection with FPS measurement
3	Evaluation on a larger AV dataset such as KITTI or BDD100K
4	Fine-tune YOLO for road objects, including auto-rickshaws and two-wheelers
5	Implement SSD and Faster R-CNN

Phase	Work
6	Controlled comparison of YOLO, SSD and Faster R-CNN on the same dataset (accuracy / FPS / latency)
7	Integrate the detector with ROS / RViz
8	Multi-sensor fusion (camera, LiDAR, radar)

15. Conclusion

This project demonstrates the use of YOLO for object detection in an autonomous-vehicle context. A Python-based system using Ultralytics and OpenCV processed three dense traffic images and produced bounding boxes and confidence scores for 61 objects, mainly buses, cars and pedestrians. Large, close vehicles were detected with high confidence (up to 0.88), whereas small, occluded and locally specific road users such as auto-rickshaws and motorcyclists were detected weakly or missed. Nearly half of the detections scored below 0.50, showing that a generic pretrained model has clear limits in complex traffic.

The study also highlights that AV perception must balance accuracy with computational speed. YOLO provides a practical framework for real-time detection and can be extended from static images to video and camera-based perception. The next steps are to measure inference latency, evaluate against labelled ground truth, fine-tune on a driving dataset, compare with SSD and Faster R-CNN under identical conditions, and integrate additional sensors and ROS.

16. References

1 Redmon, J. et al., *You Only Look Once: Unified, Real-Time Object Detection.*

2 Liu, W. et al., *SSD: Single Shot MultiBox Detector.*

3 Ren, S. et al., *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.*

4 Ultralytics YOLO Documentation.

5 KITTI Vision Benchmark Suite.

6 BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning.

7 OpenCV Documentation.